

NAT→Bridged 모드 전환

always 재시작 정책

- 틀린 healthcheck

```

rhel:
  image: nginx
  container_name: cmp-rhel
  ports:
    - "8081:80"
  volumes:
    - /opt/web/html/rhel:/usr/share/nginx/html:z
  environment:
    - OS_NAME=rhel
  restart: always
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9999"]
    interval: 5s
    timeout: 2s
    retries: 2

```

- 현재 설정

```
root@rocky days# docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cmp-rhel	nginx	"/docker-entrypoint.="	rhel	47 minutes ago	Up 45 minutes (unhealthy)	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
cmp-rocky	nginx	"/docker-entrypoint.="	rocky	17 seconds ago	Up 16 seconds (unhealthy)	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
cmp-ubuntu	nginx	"/docker-entrypoint.="	ubuntu	47 minutes ago	Up 16 seconds (healthy)	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp

```
rhel(always, unhealthy)/rocky(unless, unhealthy)/ubuntu(unless, healthy)
```

- **dockerd가 새로 시작되는 순간, 컨테이너들의 restart 정책을 다시 적용**

→ VM 재부팅 또는 도커 데몬 재시작

- unhealthy만 뜯은 상태에서 재시작 → 셋 다 재시작

```
[root@rocky days]# systemctl restart docker
[root@rocky days]# docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cmp-rhel	nginx	"/docker-entrypoint..."	rhel	48 minutes ago	Up 10 seconds (unhealthy)	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
cmp-rocky	nginx	"/docker-entrypoint..."	rocky	2 minutes ago	Up 10 seconds (unhealthy)	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
cmp-ubuntu	nginx	"/docker-entrypoint..."	ubuntu	48 minutes ago	Up 10 seconds (health: starting)	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp

- unhealthy stop → 그래도 재시작 X(unless)

```

[root@rocky days]# docker compose ps -a

```

NAME	IMAGE	SERVICE	COMMAND	STATUS	PORTS
cmp-rhel	nginx	rhel	"/docker-entrypoint..."	Up 18 seconds (unhealthy)	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
cmp-rocky	nginx	rocky	"/docker-entrypoint..."	Exited (0) 35 seconds ago	
cmp-ubuntu	nginx	ubuntu	"/docker-entrypoint..."	Up 18 seconds (healthy)	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp

- 전체 stop → always만 재시작

```
[root@rocky days]# docker compose stop rhel ubuntu
[*] stop 2/2
✓ Container cmp-rhel Stopped
✓ Container cmp-ubuntu Stopped
[root@rocky days]# systemctl restart docker
[root@rocky days]# docker compose ps -a
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cmp-rhel	nginx	"/docker-entrypoint..."	rhel	52 minutes ago	Up 5 seconds (health: starting)	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
cmp-nginx	nginx	"/docker-entrypoint..."	rocky	5 minutes ago	Exited (0) 2 minutes ago	
cmp-ubuntu	nginx	"/docker-entrypoint..."	ubuntu	52 minutes ago	Exited (0) 21 seconds ago	

- 컨테이너가 종료된 상황(종료 명령X)

```
[root@rocky day5]# docker inspect cmp-rhel | grep -i start
"Restarting": false,
"StartedAt": "2026-09-23T01:55:19.282970841Z",
"Start": "2026-09-23T10:54:54.972428037+09:00",
"Start": "2026-09-23T10:55:05.067233668+09:00",
"Start": "2026-09-23T10:55:15.119528652+09:00",
"Start": "2026-09-23T10:55:29.481604569+09:00",
"Start": "2026-09-23T10:55:39.522272761+09:00",
"RestartCount": 1,
"RestartPolicy": {
```

- always이기에 자동 재시작한 거 볼 수 있음
- docker exec cmp-rhel bash -c "kill -QUIT 1" → exit 코드가 0이기에 크래시 취급은 X
→ on-failure는 재시작 X

```

rhel:
  image: nginx
  container_name: cmp-rhel
  ports:
    - "8081:80"
  volumes:
    - /opt/web/html/rhel:/usr/share/nginx/html:z
  environment:
    - OS_NAME=rhel
  restart: always
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 10s
    timeout: 3s
    retries: 3

rocky:
  image: nginx
  container_name: cmp-rocky
  ports:
    - "8082:80"
  volumes:
    - /opt/web/html/rocky:/usr/share/nginx/html:z
  environment:
    - OS_NAME=rocky
  restart: unless-stopped
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 10s
    timeout: 3s
    retries: 3

ubuntu:
  image: nginx
  container_name: cmp-ubuntu
  ports:
    - "8083:80"
  volumes:
    - /opt/web/html/ubuntu:/usr/share/nginx/html:z
  environment:
    - OS_NAME=ubuntu
  restart: on-failure
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 10s
    timeout: 3s
    retries: 3

```

재시작 정책 각각 다르게 설정

■ 실제 검증

```

[root@rocky days]# vi /opt/days/compose.yaml
[root@rocky days]# docker exec cmp-rhel bash -c "kill -QUIT 1"
[root@rocky days]# docker exec cmp-rocky bash -c "kill -QUIT 1"
[root@rocky days]# docker exec cmp-ubuntu bash -c "kill -QUIT 1"
[root@rocky days]# docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cmp-rhel	nginx	"/docker-entrypoint..."	rhel	16 minutes ago	Up 21 seconds (healthy)	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
cmp-rocky	nginx	"/docker-entrypoint..."	rocky	16 minutes ago	Up 14 seconds (healthy)	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp

```

[root@rocky days]# docker inspect cmp-rhel | grep -i start
"Restarting": false,
"StartedAt": "2026-09-23T02:03:18.421305348Z",
"Start": "2026-09-23T11:03:01.318568573+09:00",
"Start": "2026-09-23T11:03:11.358233702+09:00",
"Start": "2026-09-23T11:03:20.604787958+09:00",
"Start": "2026-09-23T11:03:38.654270679+09:00",
"Start": "2026-09-23T11:03:48.702967379+09:00",
"RestartCount": 2,
"RestartPolicy": {
[root@rocky days]# docker inspect cmp-rocky | grep -i start
"Restarting": false,
"StartedAt": "2026-09-23T02:03:24.840920445Z",
"Start": "2026-09-23T11:03:07.01869227+09:00",
"Start": "2026-09-23T11:03:17.059958685+09:00",
"Start": "2026-09-23T11:03:35.007193841+09:00",
"Start": "2026-09-23T11:03:45.068036856+09:00",
"Start": "2026-09-23T11:03:55.11805277+09:00",
"RestartCount": 1,
"RestartPolicy": {
[root@rocky days]# docker inspect cmp-ubuntu | grep -i start
"Restarting": false,
"StartedAt": "2026-09-23T02:02:20.432040143Z",
"Start": "2026-09-23T11:02:50.705501489+09:00",
"Start": "2026-09-23T11:03:00.742695497+09:00",
"Start": "2026-09-23T11:03:10.776332934+09:00",
"Start": "2026-09-23T11:03:20.810277913+09:00",
"Start": "2026-09-23T11:03:30.856476577+09:00",
"RestartCount": 0,
"RestartPolicy": {

```

결국 재시작 정책은

1. 도커 엔진이 재시작했을 때
 2. 컨테이너가 종료됐을 때
- 적용된다.

Bridged 모드

- Linux Bridge와는 다른 계층
 - Linux Bridge = 여러 포트를 묶어서 같은 네트워크 세그먼트로 만들어주는 L2 이더넷 스위치
 - Docker가 리눅스 커널 안에 만든 가상 스위치
 - 컨테이너들의 veth 연결

- VMware의 Bridged 모드 (VMnet0)
 - VMware가 호스트 OS 레벨에서 만든 가상 스위치
 - VM의 가상 NIC ↔ 호스트의 진짜 물리 NIC(Wi-Fi) 연결
 - 목적: VM을 물리 네트워크에 직접 연결된 별개의 장치처럼 보이게 만들

```

1 [물리 공유기 네트워크 - 192.168.x.x]
2
3     ← ② VMware의 Bridged (VMnet0) : 물리 NIC ↔ VM의 ens160을 직접 연결
4
5 [VM 자신 - ens160, 지금은 192.168.x.x대역]
6
7     ← ① Linux Bridge (br-9cc12878a83e) : VM 내부 컨테이너들끼리 묶어줌
8
9 [컨테이너들 - cmp-rhel/rocky/ubuntu, 172.18.0.0/16]

```

	Linux Bridge	VMware Bridged
관리 주체	Docker (리눅스 커널의 bridge 모듈 이용)	VMware Workstation (호스트 OS 레벨)
존재 위치	VM 내부 의 커널 안	호스트 OS (하이퍼바이저) 레벨
연결 대상	컨테이너의 veth pair	VM의 가상 NIC ↔ 호스트의 물리 NIC
ip addr	VM 안에서 <code>br-...</code> 인터페이스로 보임	VM 안에서는 안 보임 (VM 입장에서선 그냥 네트워크 연결된 것)

NAT → Bridged

• 바꾸기 전(NAT)

```

[root@rocky day5]# ip addr show ens160
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:7c:f7:67 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 172.16.151.131/24 brd 172.16.151.255 scope global dynamic noprefixroute ens160
        valid_lft 1112sec preferred_lft 1112sec
    inet6 fe80::20c:29ff:fe7c:f767/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[root@rocky day5]# ip route show
default via 172.16.151.2 dev ens160 proto dhcp src 172.16.151.131 metric 100
172.16.151.0/24 dev ens160 proto kernel scope link src 172.16.151.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
172.18.0.0/16 dev br-abfed8d90d11 proto kernel scope link src 172.18.0.1
[root@rocky day5]# cat /etc/resolv.conf
# Generated by NetworkManager
search localdomain
nameserver 172.16.151.2
[root@rocky day5]#

```

• 바꾼 후(Bridged)

```
[root@rocky ~]# ip addr show ens160
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:7c:f7:67 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.10.30/16 brd 192.168.255.255 scope global dynamic noprefixroute ens160
        valid_lft 86168sec preferred_lft 86168sec
    inet6 fe80::20c:29ff:fe7c:f767/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[root@rocky ~]# ip route show
default via 192.168.0.1 dev ens160 proto dhcp src 192.168.10.30 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
172.18.0.0/16 dev br-abfed8d90d11 proto kernel scope link src 172.18.0.1
192.168.0.0/16 dev ens160 proto kernel scope link src 192.168.10.30 metric 100
[root@rocky ~]# cat /etc/resolv.conf
# Generated by NetworkManager
nameserver 168.126.63.1
nameserver 168.126.63.2
```

inet 줄의 dynamic → DHCP

- 브리지 모드에선 실제 물리 네트워크의 DNS 서버가 나타남 → KT의 공개 DNS 서버
- Docker 브리지들은 안 바뀜 → 컨테이너 내부는 독립적

- 처음에 Bridged로 전환했을 때 어떤 물리 어댑터에 연결할지를 Automatic으로 뒀더니 와이파이 어댑터가 아니라 다른 어댑터를 고름

→ 실제 있었던 어댑터 목록

```
PS C:\WINDOWS\system32> Get-NetAdapter | Where-Object {$_.Status -eq "Up"}

Name                           InterfaceDescription         ifIndex Status      MacAddress           LinkSpeed
----                           -
이더넷                        VirtualBox Host-Only Ethernet Adapter 19 Up        0A-00-27-00-00-13    1 Gbps
wt0                            WireGuard Tunnel              46 Up        00-00-00-00-00-00    100 Gbps
VMware Network Adapte...8 VMware Virtual Ethernet Adapter for VM... 15 Up        00-50-56-C0-00-08    100 Mbps
VMware Network Adapte...1 VMware Virtual Ethernet Adapter for VM... 14 Up        00-50-56-C0-00-01    100 Mbps
Wi-Fi                          Intel(R) Wi-Fi 6 AX201 160MHz      3 Up        CC-15-31-4D-18-5E    780 Mbps
```

- 발생한 문제: DHCP 실패

```
[root@rocky ~]# nmcli connection down ens160
Error: 'ens160' is not an active connection.
Error: no active connection provided.
[root@rocky ~]# nmcli connection up ens160
ip addr
Error: Connection activation failed: IP configuration could not be reserved (no available address, timeout, etc.)
Hint: use 'journalctl -xe NM_CONNECTION=b4721755-19f8-3667-9503-fad356f485e3 + NM_DEVICE=ens160' to get more details.
[root@rocky ~]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:7c:f7:67 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
```

IP를 예약할 수 없음 → NetworkManager가 DHCP 요청을 시도했다가 실패

- ens160에 inet 줄 없이 IPv6 링크로컬만 있음

- 1 IPv6 링크로컬 주소(fe80::...)는 DHCP나 외부 서버 없이,
- 2 인터페이스가 켜지지만 하면 커널이 자동으로 스스로 만들어 붙이는 주소
- 3 → 이걸 그냥 자체 생성되는 것
- 4
- 5 반면 IPv4 주소는
- 6 DHCP 서버에게 요청해서 받아와야만 붙는 주소

DHCP(Dynamic Host Configuration Protocol)

네트워크에 새로 연결된 장치에게 **IP 주소를 자동으로 나눠주는 프로토콜**

동작 과정

- ① **Discover**: 새 장치가 IP 요청하고 네트워크 전체에 브로드캐스트
- ② **Offer**: DHCP 서버(보통 공유기)가 IP 제안
- ③ **Request**: 장치가 확정 요청
- ④ **Ack**: 서버가 확정 최종 응답

```
[root@rocky ~]# tcpdump -i ens160 port 67 or port 68 -nn -v
dropped privs to tcpdump
tcpdump: listening on ens160, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:37:36.620345 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto UDP (17), length 317)
  0.0.0.0.68 > 255.255.255.255.67: BOOTP/DHCP, Request from 00:0c:29:7c:f7:67, length 289, xid 0x1
  ecedc06, secs 1, Flags [none]
    Client-Ethernet-Address 00:0c:29:7c:f7:67
    Vendor-rfc1048 Extensions
      Magic Cookie 0x63825363
      DHCP-Message (53), length 1: Request
      Client-ID (61), length 7: ether 00:0c:29:7c:f7:67
      Parameter-Request (55), length 17:
        Subnet-Mask (1), Time-Zone (2), Domain-Name-Server (6), Hostname (12)
        Domain-Name (15), MTU (26), BR (28), Classless-Static-Route (121)
        Default-Gateway (3), Static-Route (33), YD (40), YS (41)
        NTP (42), Unknown (119), Classless-Static-Route-Microsoft (249), Unknown (252)
        RP (17)
      MSZ (57), length 2: 576
      Requested-IP (50), length 4: 192.168.10.30
      Hostname (12), length 5: "rocky"
15:37:36.666081 IP (tos 0x0, ttl 64, id 0, offset 0, flags [none], proto UDP (17), length 576)
  192.168.0.1.67 > 255.255.255.255.68: BOOTP/DHCP, Reply, length 548, xid 0x1ecedc06, Flags [Broad
  cast]
    Your-IP 192.168.10.30
    Client-Ethernet-Address 00:0c:29:7c:f7:67
    Vendor-rfc1048 Extensions
      Magic Cookie 0x63825363
      DHCP-Message (53), length 1: ACK
      Server-ID (54), length 4: 192.168.0.1
      Lease-Time (51), length 4: 86400
      Subnet-Mask (1), length 4: 255.255.0.0
      Default-Gateway (3), length 4: 192.168.0.1
      Domain-Name-Server (6), length 8: 168.126.63.1,168.126.63.2

2 packets captured
5 packets received by filter
0 packets dropped by kernel
root@rocky ~#
```

- 공유기는 **MAC 주소** 기준으로 "이 장치가 이전에 쓰던 IP"를 기억 → Request부터 시작

• NAT 모드

- VMware의 가상 DHCP 서버(vmnet8)가 172.16.151.131을 임대해줌
- resolv.conf에서 봤던 dhcp 관련 정보(# Based on host file (legacy) 위쪽)가 결과물

• Bridged 모드

- VM이 물리 네트워크(공유기)의 진짜 DHCP 서버에게 새로 요청
- Automatic 어댑터 설정이 잘못되어 이 요청 자체가 공유기까지 도달 못 함
- IP configuration could not be reserved (timeout) 에러
- Wi-Fi로 수동 지정 후 요청이 제대로 도달
- 정상적으로 IP 임대받음

DHCP로 받은 IP는 영구적인 게 아니라 일정 시간(lease time) 빌려 쓰는 것

→ valid_lft(valid lifetime)로 남은 시간 확인 가능

• DHCP가 주는 것

- 1 DHCP Offer/Ack 안에 담긴 정보(대표적인 것들):
- 2 - IP 주소: 172.16.151.131
- 3 - 서브넷 마스크: 255.255.255.0

4	- 기본 게이트웨이:	172.16.151.2	← Option 3 (Router)
5	- DNS 서버:	172.16.151.2	← Option 6 (Domain Name Server)
6	- 임대 시간(lease):	1800초	

다른 기기에서 접속

- 실제 같은 공유기에 연결된 휴대폰으로 테스트

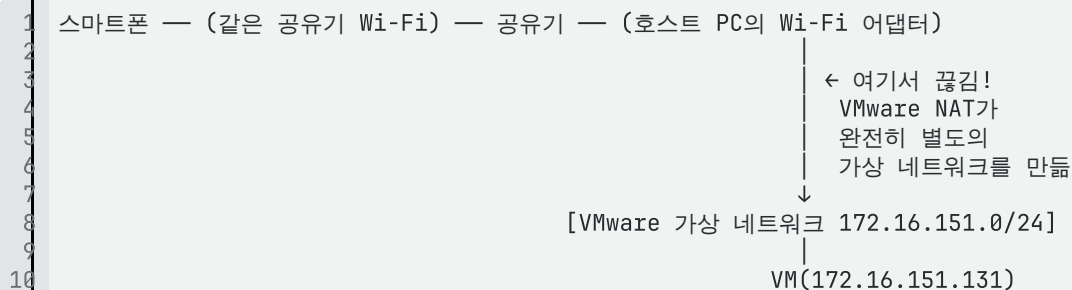


- 성공한 이유

1 스마트폰 — (같은 공유기 Wi-Fi) — 공유기 — (Bridge) — VM(192.168.10.30)

- 브리지 모드에서 VM은 물리 공유기에 꽂힌 실제 하나의 장치처럼 동작
→ 같은 공유기 → 같은 물리 네트워크 세그먼트 안에 있음 → 서로 통신 가능

- 왜 NAT일 때는 안 됐는지



- NAT 모드에선 VMware가 **host PC 안에만 존재하는 가상의 네트워크**를 만들
→ 172.16.151.0/24는 실제 물리적으로 존재하지 않음
- NAT(Network Address Translation)의 동작이 2가지(SNAT/DNAT)

- **DNAT**은 별도의 포트포워딩 설정을 해 줘야 함

단, 설정해 줘도 휴대폰에서 VM IP를 입력해서 접속은 못 함

1 ① 스마트폰 → 호스트 PC의 실제 IP: 8081 (물리 네트워크 이동)
2 ↓
3

```

3 ② VMware의 172.16.151.2에서 DNAT 발동
4   목적지: 호스트PC:8081 → 172.16.151.131:8081 (VM)      (1차 DNAT - VMware 계층)
5   ↓
6 ③ 패킷이 vmnet8을 통해 VM의 ens160으로 전달
7   ↓
8 ④ VM 안의 iptables(PREROUTING)에서 두 번째 DNAT 발동
9   목적지: 172.16.151.131:8081 → 172.18.0.3:80 (컨테이너) (2차 DNAT - Docker 계층)
10  ↓
11 ⑤ day5_default 브리지 → veth → 컨테이너 도착

```

- 1차 DNAT은 VMware의 Network Editor에서 Host Port를 추가해 줘야 함
- 2차 DNAT은 컨테이너 만들 때 -p로 설정한 것임(**8081:80**)
- SNAT은 기본적으로 동작함

NAT는 이렇게 여러 겹 쌓을 순 있지만, 서비스 개수만큼 규칙이 늘어나 Bridge보다 번거로움

- 쿠버네티스와 매핑

```

1 외부 클라이언트 → Node의 IP:NodePort      (1차 - kube-proxy가 관리하는 DNAT/IPVS)
2                  → Service의 ClusterIP:Port (2차 - Service→Pod 분산)
3                  → Pod의 실제 IP:Port      (최종 도착)

```

- **NAT 모드:** VM을 위한 별도의 사설 네트워크를 만들고, 호스트가 그 네트워크의 게이트웨이 겸 주소변환기 역할을 하는 방식
- **Bridge 모드:** 주소 변환 없이, VM을 물리 네트워크에 자기 고유 **MAC IP**를 가진 독립된 장치로 직접 참여시키는 방식

항목	NAT	Bridge
VM의 IP 출처	VMware가 관리하는 가상 네트워크(vmnet8)의 사설 대역	물리 네트워크 공유기의 DHCP가 직접 할당
게이트웨이	VMware 가상 라우터 (172.16.151.2)	물리 공유기의 IP (192.168.0.1)
DNS	게이트웨이와 동일	KT 공개 DNS
외부(같은 LAN) 기기에서 직접 접근	불가 (VM이 물리 네트워크에 자기 IP로 존재하지 않음)	가능 (VM이 물리 네트워크의 독립된 장치처럼 보임)
호스트(하이퍼바이저 PC)만 접근 가능한가	예, 기본적으로 호스트만 직접 접근 가능 (포트	아니오, 같은 네트워크 누구나 접근 가능

✔ 예전 미니pc 프로젝트 당시 포트포워딩이 안 돼서 **cloudflare tunnel**을 사용했었는데 당시 흐름이 명확히 잡힘

1. 원격으로 접속하기 위해선 미니 pc와 연결된 공유기에 포트포워딩 규칙을 설정해야 했음
2. 하지만, 규칙을 설정하려면 공유기 관리자 페이지에서 설정해야 함 → 비밀번호 모름
3. Cloudflare Tunnel로 우회 접속
 - 미니pc가 먼저 Cloudflare 서버로 아웃바운드 연결(이건 SNAT이라 설정 필요 없음)
 - tunnel로 연결 유지
 - 외부 사용자가 cloudflare 도메인으로 접속
 - 유지되고 있는 연결을 거꾸로 타고 미니pc까지 요청 전달